

Week-2 [Day-1]

Classroom Assignment

1) ICEPOT-Selenium-Java-Script-Generation.txt

Prompt used

Instructions:

- Generate a Selenium Java test script using TestNG for the login functionality
- [STRICT] Use Page Object Model design pattern
- [MANDATORY] @BeforeMethod/@AfterMethod for WebDriver setup/teardown
- [MANDATORY] Explicit waits (WebDriverWait) — no Thread.sleep
- [CRITICAL] Cover both positive and negative test scenarios
- TestNG assertions with clear failure messages

Context:

E-commerce web app login page — username/password fields, "Remember me" checkbox, "Forgot password" link; runs on Chrome, must support headless execution via TestNG XML.

Example:

@Test: testValidLogin()

Enter valid username & password, click Login

Assert redirect to /dashboard URL

Assert welcome banner text contains logged-in user's name

Persona:

- Generate: SDET skilled in Selenium WebDriver, Java, TestNG, and Page Object Model
- Review: Automation Lead, reviewing code quality and framework standards

Output Format:

Complete Java classes — Page Object class + TestNG test class, ready to paste into an IDE, plus a TestNG XML snippet for execution.

Tone:

[STRICT] Clean, production-quality code with meaningful comments.

Chatgpt response:

Below is a production-style Selenium + Java + TestNG implementation using **Page Object Model**, `@BeforeMethod/@AfterMethod`, `WebDriverWait`, and both positive and negative scenarios. It assumes Selenium 4 and a `ChromeDriver` available through Selenium Manager.

1. LoginPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

/**
 * Page Object for the E-commerce Login page.
 *
 * Responsibilities:
 * - Store page locators
 * - Perform login-page actions
 * - Expose page state required by tests
 *
 * Test assertions should remain in the TestNG test class.
 */
public class LoginPage {

    private final WebDriver driver;
    private final WebDriverWait wait;

    // Page URL
    private static final String LOGIN_URL = "https://your-ecommerce-app.com/login";

    // Locators
    private final By usernameInput = By.id("username");
    private final By passwordInput = By.id("password");
    private final By rememberMeCheckbox = By.id("rememberMe");
    private final By loginButton = By.cssSelector("button[type='submit']");
    private final By forgotPasswordLink = By.linkText("Forgot password");

    // Example application messages
    private final By loginErrorMessage = By.cssSelector(".login-error");
```

```
private final By welcomeBanner = By.cssSelector(".welcome-banner");

public LoginPage(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
}

/**
 * Navigates to the login page.
 */
public void open() {
    driver.get(LOGIN_URL);

    wait.until(ExpectedConditions.visibilityOfElementLocated(usernameInput));
}

/**
 * Enters username.
 */
public void enterUsername(String username) {
    WebElement usernameField =
        wait.until(ExpectedConditions.visibilityOfElementLocated(usernameInput));

    usernameField.clear();
    usernameField.sendKeys(username);
}

/**
 * Enters password.
 */
public void enterPassword(String password) {
    WebElement passwordField =
        wait.until(ExpectedConditions.visibilityOfElementLocated(passwordInput));

    passwordField.clear();
    passwordField.sendKeys(password);
}

/**
 * Selects the Remember Me checkbox if it is not already selected.
 */
```

```
public void selectRememberMe() {
    WebElement checkbox =
        wait.until(ExpectedConditions.elementToBeClickable(rememberMeCheckbox));

    if (!checkbox.isSelected()) {
        checkbox.click();
    }
}
```

```
/**
 * Clicks the Login button.
 */
public void clickLogin() {
    wait.until(ExpectedConditions.elementToBeClickable(loginButton)).click();
}
```

```
/**
 * Performs a standard login.
 */
public void login(String username, String password) {
    enterUsername(username);
    enterPassword(password);
    clickLogin();
}
```

```
/**
 * Performs login with Remember Me enabled.
 */
public void loginWithRememberMe(String username, String password) {
    enterUsername(username);
    enterPassword(password);
    selectRememberMe();
    clickLogin();
}
```

```
/**
 * Returns whether the Remember Me checkbox is selected.
 */
public boolean isRememberMeSelected() {
    return wait.until(
        ExpectedConditions.visibilityOfElementLocated(rememberMeCheckbox)
    );
}
```

```
        ).isSelected();
    }

    /**
     * Returns the login error message.
     */
    public String getLoginErrorMessage() {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(loginErrorMessage)
        ).getText();
    }

    /**
     * Returns the welcome banner text.
     */
    public String getWelcomeBannerText() {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(welcomeBanner)
        ).getText();
    }

    /**
     * Returns whether the Forgot Password link is displayed.
     */
    public boolean isForgotPasswordLinkDisplayed() {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(forgotPasswordLink)
        ).isDisplayed();
    }

    /**
     * Clicks the Forgot Password link.
     */
    public void clickForgotPassword() {
        wait.until(ExpectedConditions.elementToBeClickable(forgotPasswordLink)).click();
    }
}
```

2. LoginTest.java

```
package tests;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import pages.LoginPage;

/**
 * TestNG test class for E-commerce Login functionality.
 *
 * Covers:
 * - Valid login
 * - Invalid username/password
 * - Empty credentials
 * - Remember Me functionality
 * - Forgot Password link availability
 *
 * Uses:
 * - Selenium WebDriver
 * - Chrome
 * - TestNG
 * - Page Object Model
 * - Explicit waits
 */
public class LoginTest {

    private WebDriver driver;
    private WebDriverWait wait;
    private LoginPage loginPage;

    // Test data
    private static final String VALID_USERNAME = "testuser@example.com";
    private static final String VALID_PASSWORD = "ValidPassword123";
```

```
private static final String INVALID_USERNAME = "invalid@example.com";
private static final String INVALID_PASSWORD = "WrongPassword123";

private static final String LOGIN_URL =
    "https://your-ecommerce-app.com/login";

/**
 * Creates a new browser instance before every test.
 *
 * Headless execution can be controlled using:
 *
 * mvn test -Dheadless=true
 *
 * or through TestNG XML / JVM properties.
 */
@BeforeMethod(alwaysRun = true)
public void setUp() {

    ChromeOptions options = new ChromeOptions();

    String headless = System.getProperty("headless", "false");

    if (Boolean.parseBoolean(headless)) {
        options.addArguments("--headless=new");
    }

    options.addArguments("--window-size=1920,1080");
    options.addArguments("--disable-gpu");
    options.addArguments("--no-sandbox");
    options.addArguments("--disable-dev-shm-usage");

    driver = new ChromeDriver(options);

    wait = new WebDriverWait(driver, Duration.ofSeconds(10));

    loginPage = new LoginPage(driver);

    loginPage.open();
}

/**
```

```

* Valid login scenario.
*
* Expected:
* 1. User can log in with valid credentials.
* 2. User is redirected to /dashboard.
* 3. Welcome banner contains the logged-in user's name.
*/
@Test
public void testValidLogin() {

    loginPage.login(VALID_USERNAME, VALID_PASSWORD);

    // Wait until the dashboard URL is reached.
    boolean dashboardLoaded = wait.until(
        driver -> driver.getCurrentUrl().contains("/dashboard")
    );

    Assert.assertTrue(
        dashboardLoaded,
        "Login failed: User was not redirected to the dashboard."
    );

    Assert.assertTrue(
        driver.getCurrentUrl().contains("/dashboard"),
        "Unexpected URL after login. Expected URL to contain '/dashboard'."
    );

    String welcomeMessage = loginPage.getWelcomeBannerText();

    Assert.assertTrue(
        welcomeMessage.contains("testuser"),
        "Welcome banner should contain the logged-in user's name, " +
        "but actual text was: " + welcomeMessage
    );
}

/**
* Negative scenario:
* Invalid username and password should not allow login.
*/
@Test

```

```

public void testInvalidLogin() {

    loginPage.login(INVALID_USERNAME, INVALID_PASSWORD);

    String actualErrorMessage = loginPage.getLoginErrorMessage();

    Assert.assertTrue(
        actualErrorMessage.toLowerCase().contains("invalid"),
        "Expected an invalid-credentials error message, " +
        "but actual message was: " + actualErrorMessage
    );

    Assert.assertFalse(
        driver.getCurrentUrl().contains("/dashboard"),
        "User should not be redirected to the dashboard with invalid credentials."
    );
}

/**
 * Negative scenario:
 * Empty username and password should not allow login.
 */
@Test
public void testLoginWithEmptyCredentials() {

    loginPage.clickLogin();

    String currentUrl = driver.getCurrentUrl();

    Assert.assertFalse(
        currentUrl.contains("/dashboard"),
        "User should not reach the dashboard with empty credentials."
    );
}

/**
 * Negative scenario:
 * Valid username with invalid password should not allow login.
 */
@Test
public void testValidUsernameInvalidPassword() {

```

```

loginPage.enterUsername(VALID_USERNAME);
loginPage.enterPassword(INVALID_PASSWORD);
loginPage.clickLogin();

String actualErrorMessage = loginPage.getLoginErrorMessage();

Assert.assertTrue(
    actualErrorMessage.toLowerCase().contains("invalid"),
    "Expected invalid-password error message, " +
    "but actual message was: " + actualErrorMessage
);

Assert.assertFalse(
    driver.getCurrentUrl().contains("/dashboard"),
    "User should not reach the dashboard with an invalid password."
);
}

/**
 * Positive scenario:
 * Remember Me checkbox should be selectable.
 */
@Test
public void testRememberMeCheckbox() {

    loginPage.enterUsername(VALID_USERNAME);
    loginPage.enterPassword(VALID_PASSWORD);
    loginPage.selectRememberMe();

    Assert.assertTrue(
        loginPage.isRememberMeSelected(),
        "Remember Me checkbox should be selected after clicking it."
    );
}

/**
 * UI/functional scenario:
 * Forgot Password link should be available to users.
 */
@Test
public void testForgotPasswordLinkDisplayed() {

```

```

        Assert.assertTrue(
            loginPage.isForgotPasswordLinkDisplayed(),
            "Forgot Password link should be visible on the login page."
        );
    }

    /**
     * Cleans up the browser after every test.
     */
    @AfterMethod(alwaysRun = true)
    public void tearDown() {

        if (driver != null) {
            driver.quit();
        }
    }
}

```

3. testng.xml

This configuration supports **headless Chrome execution** without changing the Java test code.

```

<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
    "https://testng.org/testng-1.0.dtd

```

However, because the Java code above reads the headless setting from a **system property**, the cleanest approach is to use this slightly improved version of LoginTest.java:

```
@BeforeMethod(alwaysRun = true)
public void setUp() {

    ChromeOptions options = new ChromeOptions();

    String headless = System.getProperty("headless", "false");

    if (Boolean.parseBoolean(headless)) {
        options.addArguments("--headless=new");
    }

    options.addArguments("--window-size=1920,1080");
    options.addArguments("--disable-gpu");
    options.addArguments("--no-sandbox");
    options.addArguments("--disable-dev-shm-usage");

    driver = new ChromeDriver(options);
    wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    loginPage = new LoginPage(driver);

    loginPage.open();
}
```

For **TestNG XML itself** to control the value, use **@Parameters**:

```
import org.testng.annotations.Parameters;

@Parameters("headless")
@BeforeMethod(alwaysRun = true)
public void setUp(String headless) {

    ChromeOptions options = new ChromeOptions();

    if (Boolean.parseBoolean(headless)) {
        options.addArguments("--headless=new");
    }

    options.addArguments("--window-size=1920,1080");
    options.addArguments("--disable-gpu");
```

```
options.addArguments("--no-sandbox");
options.addArguments("--disable-dev-shm-usage");

driver = new ChromeDriver(options);

wait = new WebDriverWait(driver, Duration.ofSeconds(10));

loginPage = new LoginPage(driver);

loginPage.open();
}
```

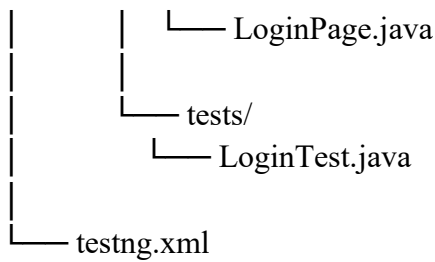
Then the XML controls execution:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
    "https://testng.org/testng-1.0.dtd
```

Recommended project structure

```
selenium-testng-project/
|
├── src/
|   ├── test/
|   │   ├── java/
|   │   └── pages/
```



Important: Replace <https://your-ecommerce-app.com/login> and the CSS/ID locators with the actual application's URL and HTML locators.

The key framework standards are satisfied: **POM separation, TestNG lifecycle hooks, explicit WebDriverWait, no Thread.sleep(), positive + negative coverage, meaningful assertions, Chrome headless support, and TestNG XML execution.**

2) ICEPOT-Bug Reporting Dashboard-HTML Output.txt

prompt used

Instructions:

- Generate a Bug Reporting Dashboard as a single self-contained HTML file
- [STRICT] No external dependencies — all CSS/JS inline
- [MANDATORY] Totals: open/closed, severity (P0–P3), bugs by module & assignee
- [CRITICAL] One chart (bar/pie) for severity distribution
- [MANDATORY] Sortable/filterable bug table (ID, Title, Severity, Status, Assignee, Module)
- Responsive layout for desktop and tablet

Context:

QA team tracks bugs across multiple projects and sprints in a spreadsheet today; needs a single-page visual dashboard to review status and trends without a backend.

Example:

Bug ID: BUG-1042 Severity: P1

Title: Login button unresponsive on Safari

Status: Open Assignee: Priya Module: Auth

Persona:

- Generate: Frontend/QA Tooling Engineer skilled in building HTML/CSS/JS dashboards

- Review: QA Lead, reviewing usability and accuracy of the dashboard

Output Format:

Single .html file — summary on top, chart in the middle, bug table at the bottom.

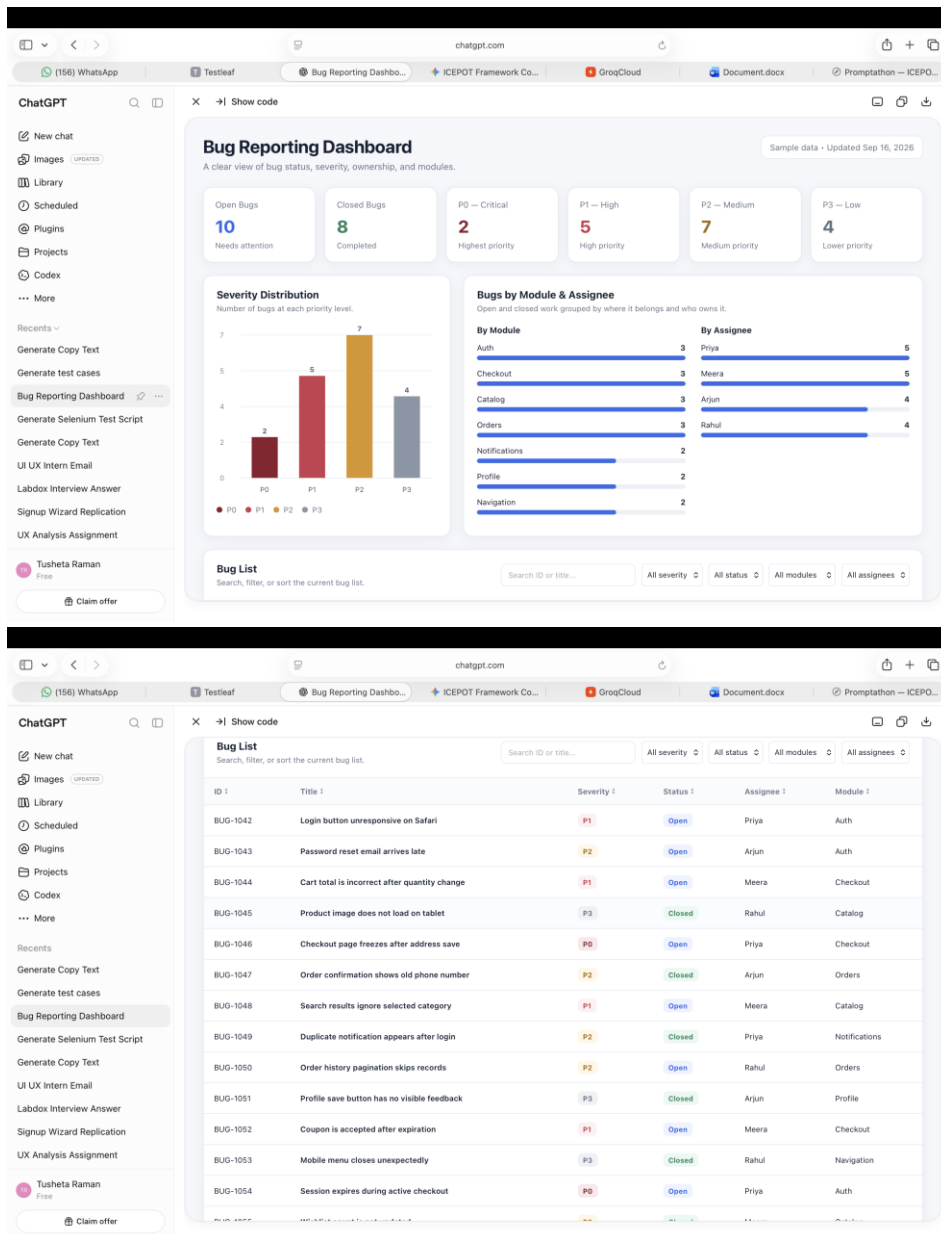
Tone:

[STRICT] Clean, professional dashboard UI copy; no jargon in labels.

Chatgpt response:

Created the self-contained Bug Reporting Dashboard with:

- Open/closed totals
- P0–P3 severity totals
- Severity bar chart
- Bugs by module
- Bugs by assignee
- Search and filters
- Sortable bug table
- Responsive desktop/tablet layout
- No external CSS/JS dependencies



3) ICEPOT-Banking Fund-Transfer Test Cases.txt

Prompt used:

Instructions:

- Fund transfer test cases within the same bank
- [STRICT] Functional only — positive and negative
- [MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data
- [CRITICAL] Mix of P0, P1, P2
- [MANDATORY] Exactly 10 test cases

- Then switch persona and review, adding comments

Context:

Banking app's fund transfer feature — transfers between two accounts within the same bank.

Example:

TC-ID: TC001 Priority: P1

Description: Verify fund transfer between two same-bank accounts succeeds

Precondition: App running; both accounts have sufficient balance

Steps: 1) Log in 2) Enter destination account 3) Enter amount & submit

Expected Result: Transfer succeeds; both balances update

Test Data: Acc 1: 445556 · Acc 2: 434343 · Amount: \$1000

Persona:

- Generate: Banking Domain QA Engineer, expert in functional testing

- Review: Team Lead, reviewing the test cases

Output Format:

10 test cases in a table, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.

Chatgpt response:

Banking Domain QA Engineer — Functional Fund Transfer Test Cases

TC ID	Priority	Description	Precondition	Steps	Expected / Actual Result	Test Data
TC001	P0	Verify successful fund transfer between two same-bank accounts	Both accounts are active; source account has sufficient balance;	1. Open Fund Transfer2. Select source account3. Enter destination account4. Enter valid	Expected: Transfer succeeds and confirmation/transaction reference is displayed. Source balance decreases and destination	Source: 445556Destination: 434343Amount: ₹1,000

TC002	P0		user is logged in	amount5. Submit transfer6. Confirm transaction	balance increases by the transfer amount. Actual: To be recorded during execution.	
		Verify transfer is rejected when source account has insufficient balance	User is logged in; source account has less balance than transfer amount	1. Open Fund Transfer2. Enter valid destination account3. Enter amount greater than available balance4. Submit	Expected: Transfer is rejected with an insufficient-balance message. No amount is debited or credited. Actual: To be recorded during execution.	Balance: ₹500Transfer: ₹1,000
		Verify transfer to a valid same-bank destination account succeeds for the minimum allowed amount	Both accounts are active; minimum transfer limit is configured	1. Open Fund Transfer2. Enter valid destination account3. Enter minimum permitted amount4. Submit and confirm	Expected: Transfer succeeds and balances are updated correctly. Actual: To be recorded during execution.	Source: 445556Destination: 434343Amount: ₹1
TC004	P1	Verify transfer is rejected when the destination account number is invalid	User is logged in; source account has sufficient balance	1. Open Fund Transfer2. Enter an invalid/non-existent same-bank account number3. Enter valid	Expected: System rejects the transfer and displays an appropriate invalid destination account message. No debit occurs. Actual:	Destination: 999999999Amount: ₹500

TC005	P1	Verify transfer is rejected when the transfer amount is zero	User is logged in; source and destination accounts are valid	amount4. Submit 1. Open Fund Transfer2. Enter valid destination account3. Enter ₹0 as amount4. Submit	To be recorded during execution. Expected: Transfer is not processed and validation indicates that the amount must be greater than zero. Actual: To be recorded during execution.	Source: 445556Destination: 434343Amount: ₹0
TC006	P1	Verify transfer is rejected when the transfer amount exceeds the permitted transaction limit	Source and destination accounts are valid; transaction limit is configured	1. Open Fund Transfer2. Enter valid destination account3. Enter amount above the allowed limit4. Submit	Expected: Transfer is rejected with an appropriate transaction-limit message. No amount is transferred. Actual: To be recorded during execution.	Daily limit: ₹1,00,000Transfer: ₹1,50,000
TC007	P1	Verify transfer is rejected when the destination account is the same as the source account	User is logged in; source account is active	1. Open Fund Transfer2. Select source account3. Enter the same account as destination4. Enter valid amount5. Submit	Expected: System prevents the self-transfer and displays an appropriate validation message. No balance changes occur. Actual: To be recorded during execution.	Source: 445556Destination: 445556Amount: ₹1,000
TC008	P2	Verify transfer is rejected when the	Source and destination	1. Open Fund Transfer2. Enter valid destination	Expected: System rejects invalid amount input and displays appropriate	Amount: ABC@100

		amount contains an invalid format	accounts are valid	account3. Enter alphabetic/special characters in amount field4. Submit	validation. Transfer is not initiated. Actual: To be recorded during execution.	
TC009	P2	Verify transfer is rejected when the destination account field is blank	User is logged in; source account is selected	1. Open Fund Transfer2. Leave destination account blank3. Enter valid amount4. Submit	Expected: System displays a mandatory-field validation message and does not process the transfer. Actual: To be recorded during execution.	Destination: BlankAmount: ₹1,000
TC010	P2	Verify transfer is rejected when the transfer amount is blank	User is logged in; source and destination accounts are valid	1. Open Fund Transfer2. Enter valid destination account3. Leave amount blank4. Submit	Expected: System displays a mandatory amount validation message and does not process the transfer. Actual: To be recorded during execution.	Source: 445556Destination: 434343Amount: Blank

Team Lead Review

Review Summary:

The test suite provides a balanced functional-only coverage of same-bank fund transfers, including successful transfers, balance validation, account validation, transaction limits, amount validation, and self-transfer prevention. The P0/P1/P2 prioritization appropriately emphasizes core transfer functionality and financial-impacting failures.

Team Lead Comments:

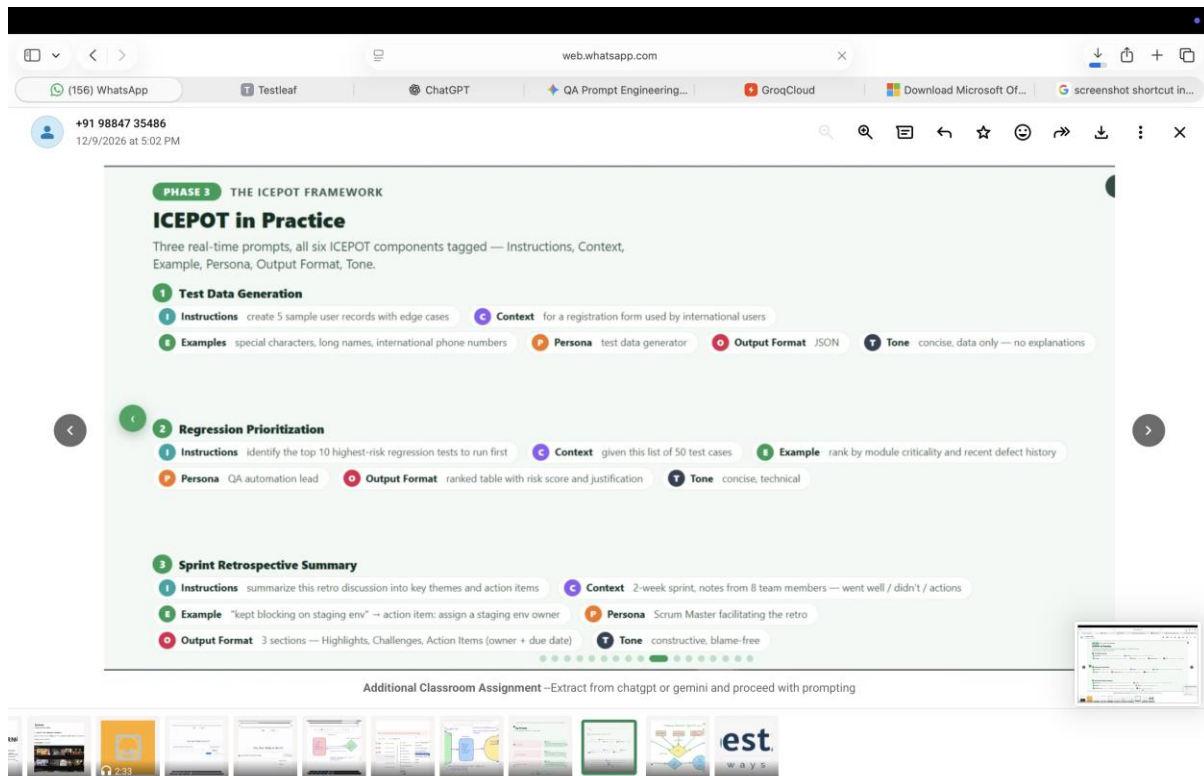
- **TC001–TC003:** Good coverage of critical successful-transfer and balance scenarios.
- **TC004:** Valid negative scenario; ensure the application does not debit the source account when destination validation fails.
- **TC005 & TC008:** Good amount-field validation coverage. Confirm the exact validation rules from the business requirements.

- **TC006:** Important financial-control scenario. Verify whether the limit is per transaction, per day, or both.
- **TC007:** Appropriate negative scenario for preventing self-transfers.
- **TC009–TC010:** Basic mandatory-field validations are correctly included.
- **Overall: Approved for functional test execution**, subject to confirmation of the application's exact transfer limits and validation messages.

Banking Domain QA Engineer — Functional Fund Transfer Test Cases						
TC ID	Priority	Description	Precondition	Steps	Expected / Actual	
TC001	P0	Verify successful fund transfer between two same-bank accounts	Both accounts are active; source account has sufficient balance; user is logged in	1. Open Fund Transfer 2. Select source account 3. Enter destination account 4. Enter valid amount 5. Submit transfer 6. Confirm transaction	Expected: Transfer succeeds and confirmation/transaction reference is displayed. Source balance decreases and destination balance increases by the transfer amount. Actual: To be recorded during execution.	
TC002	P0	Verify transfer is rejected when source account has insufficient balance	User is logged in; source account has less balance than transfer amount	1. Open Fund Transfer 2. Enter valid destination account 3. Enter amount greater than available balance 4. Submit	Expected: Transfer is rejected with an insufficient-balance message. No amount is debited or credited. Actual: To be recorded during execution.	
TC003	P0	Verify transfer to a valid same-bank destination account succeeds for the minimum allowed amount	Both accounts are active; minimum transfer limit is configured	1. Open Fund Transfer 2. Enter valid destination account 3. Enter minimum permitted amount 4. Submit and confirm	Expected: Transfer succeeds and balances are updated correctly. Actual: To be recorded during execution.	
TC004	P1	Verify transfer is rejected when the destination account number is invalid	User is logged in; source account has sufficient balance	1. Open Fund Transfer 2. Enter an invalid/non-existent same-bank account number 3. Enter valid amount 4. Submit	Expected: System rejects the transfer and displays an appropriate invalid destination account message. No debit occurs. Actual: To be recorded during execution.	
TC005	P1	Verify transfer is rejected when the transfer amount is zero	User is logged in; source and destination accounts are valid	1. Open Fund Transfer 2. Enter valid destination account 3. Enter ₹0 as amount	Expected: Transfer is not processed and validation indicates that the amount must be greater than zero. Actual: To be recorded during execution.	

Banking Domain QA Engineer — Functional Fund Transfer Test Cases				
Precondition	Steps	Expected / Actual Result	Test Data	
Both accounts are active; source account has sufficient balance; user is logged in	1. Open Fund Transfer 2. Select source account 3. Enter destination account 4. Enter valid amount 5. Submit transfer 6. Confirm transaction	Expected: Transfer succeeds and confirmation/transaction reference is displayed. Source balance decreases and destination balance increases by the transfer amount. Actual: To be recorded during execution.	Source: 445556 Destination: 434343 Amount: ₹1,000	
User is logged in; source account has less balance than transfer amount	1. Open Fund Transfer 2. Enter valid destination account 3. Enter amount greater than available balance 4. Submit	Expected: Transfer is rejected with an insufficient-balance message. No amount is debited or credited. Actual: To be recorded during execution.	Balance: ₹500 Transfer: ₹1,000	
Both accounts are active; minimum transfer limit is configured	1. Open Fund Transfer 2. Enter valid destination account 3. Enter minimum permitted amount 4. Submit and confirm	Expected: Transfer succeeds and balances are updated correctly. Actual: To be recorded during execution.	Source: 445556 Destination: 434343 Amount: ₹1	
User is logged in; source account has sufficient balance	1. Open Fund Transfer 2. Enter an invalid/non-existent same-bank account number 3. Enter valid amount 4. Submit	Expected: System rejects the transfer and displays an appropriate invalid destination account message. No debit occurs. Actual: To be recorded during execution.	Destination: 999999999 Amount: ₹500	
User is logged in; source and destination accounts are valid	1. Open Fund Transfer 2. Enter valid destination account 3. Enter ₹0 as amount	Expected: Transfer is not processed and validation indicates that the amount must be greater than zero. Actual: To be recorded during execution.	Source: 445556 Destination: 434343 Amount: ₹0	

Additional Classroom Assignment --Extract from chatgpt or gemini and proceed with prompting



Chatgpt:

Prompt used: generate copy text format foe the attached image

Response:

PHASE 3 — THE ICEPOT FRAMEWORK

ICEPOT in Practice

Three real-time prompts, all six ICEPOT components tagged — Instructions, Context, Example, Persona, Output Format, Tone.

1. Test Data Generation

Instructions: create 5 sample user records with edge cases

Context: for a registration form used by international users

Examples: special characters, long names, international phone numbers

Persona: test data generator

Output Format: JSON

Tone: concise, data only — no explanations

2. Regression Prioritization

Instructions: identify the top 10 highest-risk regression tests to run first

Context: given this list of 50 test cases

Example: rank by module criticality and recent defect history

Persona: QA automation lead

Output Format: ranked table with risk score and justification

Tone: concise, technical

3. Sprint Retrospective Summary

Instructions: summarize this retro discussion into key themes and action items

Context: 2-week sprint, notes from 8 team members — went well / didn't / actions

Example: "kept blocking on staging env" → action item: assign a staging env owner

Persona: Scrum Master facilitating the retro

Output Format: 3 sections — Highlights, Challenges, Action Items (owner + due date)

Tone: constructive, blame-free

Gemini:

Prompt: generate copy text format for the attached image

Response:

PHASE 3 THE ICEPOT FRAMEWORK

ICEPOT in Practice

Three real-time prompts, all six ICEPOT components tagged — Instructions, Context, Example, Persona, Output Format, Tone.

1 Test Data Generation

- **Instructions:** create 5 sample user records with edge cases

- **Context:** for a registration form used by international users
- **Examples:** special characters, long names, international phone numbers
- **Persona:** test data generator
- **Output Format:** JSON
- **Tone:** concise, data only — no explanations

2 Regression Prioritization

- **Instructions:** identify the top 10 highest-risk regression tests to run first
- **Context:** given this list of 50 test cases
- **Example:** rank by module criticality and recent defect history
- **Persona:** QA automation lead
- **Output Format:** ranked table with risk score and justification
- **Tone:** concise, technical

3 Sprint Retrospective Summary

- **Instructions:** summarize this retro discussion into key themes and action items
- **Context:** 2-week sprint, notes from 8 team members — went well / didn't / actions
- **Example:** "kept blocking on staging env" → action item: assign a staging env owner
- **Persona:** Scrum Master facilitating the retro
- **Output Format:** 3 sections — Highlights, Challenges, Action Items (owner + due date)
- **Tone:** constructive, blame-free
